Lab 08 — Hands-on lab: configure without rebuilding, hide without fooling yourself

Goal: drive the same `api:1.0` image through variables, measure source precedence, see with your own eyes where a secret shows up (logs, `ps`, `inspect`, `env`), then make it disappear with `podman secret` and a build-time secret mount.

Prerequisites — Labs 01 to 07 completed. Image `api:1.0` present (lab 07, step 0); `postgres:16-alpine` gets pulled again.

Files provided (`files/`) — none: everything is created at the terminal.

Step 1 — The same image, other values

```
mkdir -p ~/labo-docker/08 && cd ~/labo-docker/08
podman run -d --name api -p 18080:8080 api:1.0 ; sleep 2
curl -s localhost:18080/ ; echo
podman rm -f -t 0 api
podman run -d --name api -p 18080:8080 -p 19090:9090 -e APP_MESSAGE="config via -e" -e
APP_PROFILE=prod -e SERVER_PORT=9090 api:1.0 ; sleep 2
curl -s localhost:18080/ ; echo
curl -s localhost:19090/ ; echo
podman logs api | head -2
```

Observe first `{"message":"Bonjour depuis l'API","profile":"default"}` — the defaults coded into the application —, then **nothing** on 18080 and `{"message":"config via -e","profile":"prod"}` on 19090, with `API demarree sur le port 9090 (profil prod)` in the logs.

Explanation. Three variables changed the message, the profile and the listening port, with no rebuild. And `-p 18080:8080` relays to a port where nobody listens any more: `EXPOSE 8080` in the image checked nothing.

Step 2 — Everything is visible

```
podman inspect api --format '{{range .Config.Env}}{{println .}}{{end}}' | grep APP_
podman exec api env | grep APP_
podman exec api sh -c 'tr "\0" "\n" < /proc/1/environ | grep APP_'
podman rm -f -t 0 api
```

Observe three times the same `APP_MESSAGE=config via -e` and `APP_PROFILE=prod`: in the container's metadata, in the environment seen by `exec`, and in that of process 1.

```
podman run -d --name api api:1.0 --db-password=Secr3t ; sleep 2
podman logs api | head -1
podman inspect api --format '{{json .Config.Cmd}}'
podman top api args
ps -eo args | grep -c 'db-password=Secr3t'
podman rm -f -t 0 api
```

Observe `Arguments recus : --db-password=Secr3t` in the **logs**, `["--db-password=Secr3t"]` in the inspect output, `java -jar /app/api.jar --db-password=Secr3t` in `top`, and `2` host processes showing the value (the `java` and the `grep` itself).

Explanation. A variable is readable by anyone with engine access; an argument is readable by **everyone** on the machine, and ends up in the logs. Neither is a safe.

Step 3 — Source precedence

```
printf '# API configuration (never in git)\nAPP_MESSAGE=config via env-
file\nAPP_PROFILE=staging\n' > api.env
printf 'FROM localhost/api:1.0\nENV APP_MESSAGE="config from the image" APP_PROFILE=image\n' >
Dockerfile.env
podman build -q -f Dockerfile.env -t api:env .
podman image inspect api:env --format '{{range .Config.Env}}{{println .}}{{end}}' | grep APP_
```

Observe `APP_MESSAGE=config from the image` and `APP_PROFILE=image`: default values written into the image.

```
podman run -d -p 18080:8080 --name a1 api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ;
podman rm -f -t 0 a1
podman run -d -p 18080:8080 --name a2 --env-file api.env api:env ; sleep 2 ; curl -s
localhost:18080/ ; echo ; podman rm -f -t 0 a2
podman run -d -p 18080:8080 --name a3 -e APP_PROFILE=prod --env-file api.env api:env ; sleep 2
; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a3
APP_MESSAGE="from the host shell" podman run -d -p 18080:8080 --name a4 -e APP_MESSAGE api:env
; sleep 2 ; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a4
```

Observe in turn `"config from the image"/"image"` (the image alone), `"config via env-file"/"staging"` (the file overrides the image), `"config via env-file"/"prod"` (`-e` overrides the file, although it is written **before** `--env-file`), and `"from the host shell"/"image"` (`-e APP_MESSAGE` with no value copied the shell variable).

Explanation. Precedence is by category, not by position: `-e` > `--env-file` > `ENV`. The `api.env` file is the natural candidate for "one set of values per environment", provided it stays out of Git.

Step 4 — The JVM reads the environment too

```
podman run -d --name a5 -e JAVA_TOOL_OPTIONS="-Xmx64m" api:1.0 ; sleep 2
podman logs a5 2>&1 | head -1
podman rm -f -t 0 a5
```

Observe `Picked up JAVA_TOOL_OPTIONS: -Xmx64m`: the JVM took the option without any change to the `ENTRYPOINT`.

Explanation. This is the standard lever to tune memory or attach an agent to an image you do not rebuild.

Step 5 — An engine secret for PostgreSQL

```
printf 'S3cr3t-pg' | podman secret create db_password -
podman secret ls
podman secret inspect db_password --format '{{.Spec.Driver.Name}}
{{.Spec.Driver.Options.path}}'
podman secret inspect --showsecret db_password --format '{{.SecretData}}'
```

Observe an ID, `db_password` with the `file` driver, the path `…/storage/secrets/filedriver`, and — with `--showsecret` — `S3cr3t-pg`: stored, not encrypted.

```
podman run -d --name db --secret db_password -e POSTGRES_PASSWORD_FILE=/run/secrets/db_password
postgres:16-alpine ; sleep 4
podman exec db ls -l /run/secrets/
podman inspect db --format '{{range .Config.Env}}{{println .}}{{end}}' | grep POSTGRES
podman inspect db --format '{{json .Config.Env}}' | grep -c 'S3cr3t'
podman run --rm --network container:db -e PGPASSWORD=S3cr3t-pg postgres:16-alpine psql -h
localhost -U postgres -c 'select current_user'
```

Observe `-r--r--r-- 1 root root 9 … db_password`, a single `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` in the environment, `0` occurrences of the value in the inspect output, and `current_user = postgres`: the password really is the secret's.

```
podman rm -f -t 0 db
podman run -d --name db2 --secret db_password,type=env,target=POSTGRES_PASSWORD postgres:16-
alpine ; sleep 3
podman inspect db2 --format '{{json .Config.Env}}' | grep -c POSTGRES_PASSWORD
podman exec db2 env | grep POSTGRES_PASSWORD
podman rm -f -t 0 db2
```

Observe `0` in the inspect output but `POSTGRES_PASSWORD=S3cr3t-pg` in the process environment: the `type=env` variant serves applications that do not read files, at the price of a value visible to `exec`.

Explanation. The secret travels neither through the command line nor through the metadata. With `_FILE`, only the process that opens `/run/secrets/db_password` sees it.

Step 6 — A secret at build time

```
printf 'ghp_private-token' > token.txt
printf 'FROM docker.io/library/alpine\nRUN --mount=type=secret,id=token,required=true wc -c
/run/secrets/token && echo "token used, not kept"\nRUN ls /run/secrets 2>&1 || true\n' >
Dockerfile.secret
podman build --secret id=token,src=token.txt -f Dockerfile.secret -t secret-demo . 2>&1 | grep
-v '^-->'
podman history --no-trunc secret-demo --format '{{.CreatedBy}}' | head -2
podman run --rm secret-demo ls /run/secrets
```

Observe `17 /run/secrets/token` during the instruction that mounts the secret, then `ls: /run/secrets: No such file or directory` from the next instruction on; in the history, the line `--mount=type=secret,id=token,…` **without the value**; and in the final image, no `/run/secrets`.

```
podman build -f Dockerfile.secret -t secret-demo2 . 2>&1 | tail -1
podman build --no-cache -f Dockerfile.secret -t secret-demo2 . 2>&1 | tail -1
```

Observe that the build **without** `--secret` succeeds (everything comes from the cache), then, with `--no-cache`, `Error: … secret required but no secret with id "token" found`.

Explanation. The secret exists only during one instruction, and enters no layer. The cache asks for nothing again: an expired token is discovered on the first build without cache.

Clean-up

```
podman secret rm db_password
podman rmi api:env secret-demo secret-demo2
podman rmi docker.io/library/postgres:16-alpine
rm -f token.txt Dockerfile.secret
podman secret ls ; podman ps -a --format '{{.Names}}'
```

Observe an empty secrets list and no container. Keep `api.env` and `Dockerfile.env`: lab 09 reuses `api.env` with Compose.

What you must be able to state now

- Three `-e` options changed message, profile and port with no rebuild — and `-p 18080:8080` relayed into the void.
- A variable shows in `inspect`, `exec env` and `/proc/1/environ`; an argument shows **in addition** in the logs, `top` and the host's `ps`.
- `-e` > `--env-file` > `ENV`, whatever the order; `-e KEY` alone copies the host variable.
- `Picked up JAVA_TOOL_OPTIONS`: the JVM configures itself through a variable.
- `podman secret` + `_FILE`: `0` occurrences of the password in `inspect`, a `444` file in `/run/secrets/`; `--showsecret` proves it is not encrypted.
- `--mount=type=secret`: `17 /run/secrets/token` during the instruction, `No such file or directory` afterwards, nothing in `history`.
- Without `--secret`, the cached build succeeds; `--no-cache` → `secret required but no secret with id "token" found`.